



Bachelorarbeit

Learning-based Multirotor Control Enhancements

vorgelegt von

Eckart Cobo Briesewitz

Matrikelnr. 468840

October 16, 2024

Betreuer: Khaled Wahba & Pia Hanfeld

Prüfer: Prof.-Dr. Wolfgang Hönig & Prof.-Dr. Marc Toussaint



Intelligent Multi-Robot Coordination Lab

Fakultät IV Elektrotechnik und Informatik | Technische Universität Berlin

Contents

1	Introduction	8
2	Background	9
2.1	Quadrotor Dynamics	9
2.2	Residual Forces	9
2.3	Flight Controllers	10
2.3.1	Brescianini	10
2.3.2	INDI	10
2.3.3	Mellinger	11
2.3.4	Lee	11
2.4	Experimental Setup	11
2.5	Machine Learning	13
2.5.1	Neural Networks	13
2.5.2	Decision Tree Ensemble	14
2.6	Dynamics Learning	15
2.6.1	Parameter Estimation	15
2.6.2	Full Regression	15
2.6.3	Residual Dynamics	16
2.6.4	Observation-based Dynamics	16
3	Related Work	17
4	Methodology	19
4.1	Final Pipeline	19
4.2	Initial Experiments	20
4.3	Estimating the Residuals with Learning Algorithms	22
4.3.1	Initial Model Comparison	24
4.4	Translating the Models Into C Code	25
4.4.1	Neural Network	26
4.4.2	Decision Tree Ensemble	26
4.4.3	Standard Residual Calculation	27
4.5	Testing the Performance	28
5	Results	30
5.1	Simulation	30
5.2	Real World Flights	30
5.2.1	Standard Lee Controller	30
5.2.2	Neural Network	32
5.2.3	Decision Tree Ensemble	34
5.2.4	Standard Residual Calculation Without Learning Algorithms	36
5.3	Results on Brushless Crazyflie	38
5.3.1	Decision Tree Ensemble	38
5.3.2	Neural Network	39
5.3.3	Standard Residual Calculation	40
5.4	Results Discussion	41
6	Future Work	42

Eidesstattliche Erklärung / Statutory Declaration

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig und eigenhändig sowie ohne unerlaubte fremde Hilfe und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe.

I hereby declare that I have created this work completely on my own and used no other sources or tools than the ones listed.

Eckart Cobo Briesewitz

Berlin, October 16, 2024

Abstract

The increasing complexity of quadrotor applications has led to the need for more reliable flight controllers that can ensure stable and precise navigation. Traditional flight controllers model a large part of the aerodynamic forces acting on quadrotors, however there are still some unmodeled residual forces that do not get computed as computing them is extremely complex. Therefore there is an interest in modeling these forces with learning based algorithms that can alleviate this high complexity. In this work we concentrate on the process of uploading these learning models into quadrotor hardware, specifically the STM32 found in the Bitcraze Crazyflie. Our goal is to see if there is any improvement in high speed flight from using these learning methods to predict the residual forces. In the end we found that there is a 15% to 25% increase in performance from doing this. In addition, some useful methods were developed to upload and test the performance of future learning based methods into the Crazyflie quadrotor.

The Code is available at <https://github.com/Tupryk/BachelorThesis>

Zusammenfassung

Die steigende Komplexität von Quadrotor-Anwendungen hat zu einem Bedarf an zuverlässigeren Flugsteuerungen geführt, die eine stabile und präzise Navigation gewährleisten können. Herkömmliche Flugregler modellieren zwar einen Großteil der auf Quadrotoren wirkenden aerodynamischen Kräfte, dennoch bleiben einige nicht modellierte Restkräfte unberücksichtigt, da ihre Berechnung extrem komplex ist. Daher besteht ein zunehmendes Interesse daran, diese Kräfte mithilfe von lernenden Algorithmen zu modellieren, die diese Komplexität reduzieren können. Diese Arbeit befasst sich mit dem Prozess der Integration solcher Lernmodelle für die Quadrotor-Hardware, insbesondere dem STM32 Mikrocontroller der Bitcraze Crazyflie. Unser Ziel ist es herauszufinden, ob die Verwendung dieser Lernmethoden zur Vorhersage von Restkräften, d.h. allen nicht-modellierten aerodynamischen Kräften, das Flugverhalten bei hohen Geschwindigkeiten verbessert. Letztendlich haben wir festgestellt, dass dies zu einer Leistungssteigerung von 15% bis 25% führt. Zusätzlich, wurden einige nützliche Methoden entwickelt, um die Leistung zukünftiger lernbasierter Methoden in den Crazyflie-Quadrotor zu laden und zu testen.

Der Code ist verfügbar unter <https://github.com/Tupryk/BachelorThesis>

Nomenclature

$\mathcal{W} \in SE(3)$	World frame	—
$\mathcal{B} \in SE(3)$	Body frame (attached to the quadrotor)	—
$\mathbf{r} \in \mathbb{R}^3$	Translational component of the quadrotor's pose	$[m]$
$\mathbf{R} \in SO(3)$	Rotational component of the quadrotor's pose	—
$\mathbf{v} \in \mathbb{R}^3$	Velocity of the quadrotor in the world frame	$[m/s]$
$\boldsymbol{\omega} \in \mathbb{R}^3$	Angular velocity of the quadrotor in quadrotor frame	$[rad/s]$
$m \in \mathbb{R}$	Mass of the quadrotor	$[kg]$
$g \in \mathbb{R}$	Gravity	$9.81 [m/s^2]$
$\mathbf{u} \in \mathbb{R}^4$	Control input moments (thrust and torques)	$[u_1, u_2, u_3, u_4]$
$u_1 \in \mathbb{R}$	Total thrust	$[N]$
$u_{[2,3,4]} \in \mathbb{R}$	Body moments	$[N]$
$\mathbf{J} \in \mathbb{R}^{3 \times 3}$	Inertia matrix of the quadrotor	$[kg \cdot m^2]$
$\times$	Cross product operator	—
$L \in \mathbb{R}$	Distance from the rotor to the center of the quadrotor	$[m]$
$\hat{\psi}_i \in \mathbb{R}$	Force generated by rotor i	$[N]$

$t2t \in \mathbb{R}$	Thrust-to-torque ratio	—
$\mathbf{z}_{\mathcal{W}} \in \mathbb{R}^3$	Upward direction in the world frame	—
$\mathbf{z}_{\mathcal{B}} \in \mathbb{R}^3$	Upward direction in the body frame	—

1 Introduction

The growing complexity of quadrotor applications in challenging environments, influenced by intricate aerodynamic forces, such as for inspections, deliveries, search-and-rescue missions, or synchronized aerial displays, has necessitated the development of advanced methods for predicting their interactions with aerodynamic forces. A basic physical model alone cannot fully capture the complex dynamics of quadrotor movement. External forces like ground effect, blade flapping, or aerodynamic drag, among others, have a significant impact on quadrotor flight, especially when performing high-speed maneuvers. Therefore, models that can learn and correct these inaccuracies are essential.

Due to the difficulty of modeling these forces, methods are needed that can approximate these forces efficiently and at high frequency so that they can be used in real-time during high-speed flights. The common approach is to use learning algorithms for this purpose. To do this, flight data has to be collected. The data should cover a large variety of quadrotor state information to account for the full spectrum of potential states the quadrotor may encounter during operation. From these states, the residual forces can be calculated. With learning algorithms, we can create models that can map states to residual forces which can be used to adjust rotor force outputs in real time. Previous research has demonstrated that learning-based models can effectively approximate these residual forces, which can then be integrated into flight controllers for Unmanned Aerial Vehicles (UAVs) [25] [27] [1].

In a prior thesis [25] conducted at the IMRC Lab at TU Berlin, a variety of models were developed for this purpose, categorized into neural networks and decision tree ensembles. The models were able to accurately model the residual forces acting on a Crazyflie quadrotor. However, this was only done offline and not on the real hardware, therefore the question still remains if there would be any real improvement in flight from using these learning models to adjust rotor force outputs.

This thesis focuses on evaluating the two best-performing models from that thesis, one from each category, in real-world flights. These models will be integrated into the firmware of a Crazyflie quadrotor, and their performance will be assessed both in simulation and in real-world flight tests. We expect that this addition to the flight controller improves the flight performance at high speeds.

2 Background

In this thesis, we use Crazyflie quadrotors for our experiments. The Crazyflie quadrotors are developed by Bitcraze, a company based in Sweden. The most popular model, the Crazyflie 2.1, is a small quadrotor often used for research purposes, with an arm length of 4.6cm and 34.7g weight [7]. It is operated by an STM32 microprocessor, specifically the STM32F405. This microprocessor has a clock speed of 168 MHz, 1 MB of flash memory, and 192 KB of RAM.

2.1 Quadrotor Dynamics

The rotation and translation of the quadrotor will be defined relative to a fixed world frame. We can define the world frame as $\mathcal{W}$ and the pose of the quadrotor will be defined by the frame $\mathcal{B}$. Both $\mathcal{W}$ and $\mathcal{B}$ are a part of SE(3), the Special Euclidean group, which represents all possible positions and rotations in three-dimensional space. Therefore we can divide the pose of the quadrotor into a rotational component $\mathbf{R}$ and a translational component $\mathbf{r}$. The quadrotor's state also includes the global velocity $\mathbf{v}$ and the angular velocity $\boldsymbol{\omega}$. The following equations will be relevant for calculating the quadrotor's dynamics [28]:

$$\dot{\mathbf{r}} = \mathbf{v} \quad (1)$$

$$m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ u_1 \end{bmatrix} \quad (2)$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} \quad (3)$$

Where $\dot{\mathbf{v}}$ and $\dot{\mathbf{r}}$ are the derivatives of the velocity and angular velocity respectively, m is the mass of the multirotor, $\mathbf{g} = [0 \ 0 \ -g]^T$ represents gravity, $u_{[2,3,4]}$ are the body moments and u_1 is the total thrust, $\mathbf{J}$ represents the inertia matrix for the quadrotor, and $\times$ is the cross product. In order to calculate the $\mathbf{u} = [u_1 \ u_2 \ u_3 \ u_4]$ vector we can use the following equation [27] [18] [17]:

$$\mathbf{u} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -L & -L & L & L \\ -L & L & L & -L \\ -t2t & t2t & -t2t & t2t \end{bmatrix} \begin{bmatrix} \hat{\psi}_1 \\ \hat{\psi}_2 \\ \hat{\psi}_3 \\ \hat{\psi}_4 \end{bmatrix} \quad (4)$$

Where L represents the distance of a rotor to the center of the quadrotor, $t2t$ is the thrust-to-torque ratio and $\hat{\psi}_i$ represents as the force of the rotor i .

2.2 Residual Forces

To enhance the quadrotor's flight performance, we aim to calculate the residual forces that affect its movement. These forces can arise from various sources such as blade flapping, ground effect when flying close to the ground, or downwash from nearby quadrotors. Because these forces are complex and variable, they cannot be easily captured by standard dynamics equations and would require intricate physical simulations to be approximated. The residual forces can be represented as two 3D vectors: $\mathbf{f}_a$, which corresponds to the translational forces, and $\boldsymbol{\tau}_a$, representing the rotational forces or torques. By revisiting the dynamics equations, we can define the residual forces as additional forces that supplement the existing model [28].

$$m\dot{\mathbf{v}} = m\mathbf{g} + \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ u_1 \end{bmatrix} + \mathbf{f}_a \quad (5)$$

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} + \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} + \boldsymbol{\tau}_a \quad (6)$$

From this we can then calculate the residual forces and torques like follows:

$$\mathbf{f}_a = m\dot{\mathbf{v}} - m\mathbf{g} - \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ u_1 \end{bmatrix} \quad (7)$$

$$\boldsymbol{\tau}_a = \mathbf{J}\dot{\boldsymbol{\omega}} - \mathbf{J}\boldsymbol{\omega} \times \boldsymbol{\omega} - \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} \quad (8)$$

2.3 Flight Controllers

The flight controller of a quadrotor is responsible for stabilizing and controlling its movements. Given the non-linear and underactuated nature of quadrotors, effective control is essential for stable flight. In addition, flight controllers enable autonomy, allowing quadrotors to perform precise maneuvers and respond to dynamic conditions.

By processing data from various sensors, the flight controller adjusts the thrust output of each rotor to manage the drone's position, orientation, and velocity. It continually adapts to external disturbances, ensuring stable flight. As a result, multiple control algorithms have been developed to enhance the performance of quadrotors across various flight scenarios. Following are some prominent geometric quadrotor controllers that rely on differential flatness.

2.3.1 Brescianini

The Brescianini controller [3] is relatively simple due to its straightforward design, yet it is still capable of achieving stable flight. It leverages geometric control principles to maintain the desired flight trajectory with minimal computational complexity. The simplicity arises from its clear separation of rotational and translational dynamics, allowing these two components to be controlled independently. This decoupling simplifies the control problem, making it easier to implement and tune while still providing robust performance for stable and accurate flight control.

2.3.2 INDI

The Incremental Nonlinear Dynamic Inversion (INDI) controller [30] is designed to enhance the handling of a quadrotor's inherently non-linear dynamics, offering improvements over traditional linear control methods. Unlike linear controllers, which struggle with dynamic changes and external disturbances, INDI computes control inputs incrementally using real-time IMU data. This enables it to react more swiftly to changes in the system's dynamics, disturbances, or model uncertainties, ensuring more precise and responsive control even in highly dynamic environments.

2.3.3 Mellinger

The Mellinger controller [18] is a simplified version of the Lee controller, where some higher-order terms are ignored. It is known for its ability to perform aggressive maneuvers while maintaining control stability. The basis of this controller is differential flatness, which allows for the decoupling of position and attitude control. This controller can deal with sharp turns or rapid accelerations while staying stable.

2.3.4 Lee

The Lee controller [14] employs geometric control theory to provide globally stable and robust control of a quadrotor's attitude and position. It is designed to work on the manifold of $SO(3)$ (the special orthogonal group for 3D rotations), which makes it particularly effective in avoiding issues related to singularities or gimbal lock that other controllers may encounter. The control inputs given by the Lee controller are defined as follows:

$$\mathbf{f} = -(-k_{\mathbf{r}}e_{\mathbf{r}} - k_{\mathbf{v}}e_{\mathbf{v}} - mg\mathbf{z}_{\mathcal{W}} + m\ddot{\mathbf{r}}_d) \cdot \mathbf{R}e_3 \quad (9)$$

$$\boldsymbol{\tau} = -k_{\mathbf{R}}e_{\mathbf{R}} - k_{\boldsymbol{\omega}}e_{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} - \mathbf{J}(\dot{\boldsymbol{\omega}}\mathbf{R}^T\mathbf{R}_d\boldsymbol{\omega}_d - \mathbf{R}^T\mathbf{R}_d\dot{\boldsymbol{\omega}}_d) \quad (10)$$

Where $k_{\mathbf{r}}$, $k_{\mathbf{v}}$, $k_{\mathbf{R}}$ and $k_{\boldsymbol{\omega}}$ are some positive constants, $e_3 = [0 \ 0 \ 1]^T$, $\mathbf{f}$ is the total thrust and $\boldsymbol{\tau} = [u_2 \ u_3 \ u_4]^T$ is the control moment which corresponds to the tracking controller on $SO(3)$. The vector $\mathbf{v}$ is the velocity of the center of mass in the inertial frame, and is equal to $\dot{\mathbf{r}}$. The tracking errors for $\mathbf{r}$, $\mathbf{v}$, $\mathbf{R}$ and $\boldsymbol{\omega}$ are represented by $e_{\mathbf{r}}$, $e_{\mathbf{v}}$, $e_{\mathbf{R}}$ and $e_{\boldsymbol{\omega}}$. The acceleration of the center of mass in the inertial frame is represented by $\ddot{\mathbf{r}}$ and the angular acceleration is represented by $\dot{\boldsymbol{\omega}}$. The d index represents the desired state of the value, for example $\mathbf{R}_d$ is the desired rotation. The **hat map** $\hat{\cdot} : \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ is defined by the condition that for all $x, y \in \mathbb{R}^3$, $\hat{x}y = x \times y$.

2.4 Experimental Setup



Figure 1: Crazyflie quadrotor



Figure 2: Crazyflie quadrotor with motion capture marker and SD card reader

Although the Crazyflie is equipped with sensors, including a gyroscope, some of the flight data, such as position and velocity estimates, were obtained using a motion capture system for a more precise state estimate. The motion capture system provides higher accuracy by directly measuring the drone's position in 3D space, minimizing the drift and noise typically associated with onboard

sensor measurements like those from IMUs. Specifically, an OptiTrack setup was used, where a small near-infrared reflective marker was attached to the Crazyflie, as illustrated in Figure 2. There are a total of 14 cameras which are interfaced with the motive application. The OptiTrack system runs on a Windows computer, which broadcasts the motion capture data over the network. This data is then integrated into the control system using ROS (Robot Operating System) [20] to process and relay the position and velocity estimates to the Crazyflie in real-time, enabling precise feedback control during flight.

This same flight tracking system was also employed to evaluate the improved flight performance of the model. An SD card reader deck was mounted on the Crazyflie in order to collect the relevant data needed for calculating the residual forces, as will be explained in 2.2, as well as data relating to the learning model inference for testing.

The initial testing was done on a Crazyflie 2.1 and eventually the same models were trained on a brush-less motor version of this quadrotor 3 which have less motor delay, that means, the motors react faster to input signals. It is important that the model that is used on a specific drone is trained on the residual calculations for the corresponding drone, this means, a model trained on a Crazyflie 2.1 will only work on other Crazyflies 2.1. This is because the residual calculations are linked to properties like it's weight. While these models will only work on the specific quadrotor configuration that they were trained on, the methods used to develop them will be transferable to a wider range of multi-rotors.



Figure 3: Crazyflie with brush-less motors and IR deck.

As shown in Figure 3, the quadrotor has no motion capture marker, this is because it is equipped with an infrared (IR) deck. This deck emits infrared light that can be used to track the Crazyflie with the motion capture setup. For this to work correctly, the motion capture cameras need to be set to only receive infrared signals, as in this case infrared light is being emitted, rather than having a passive marker, like in 2, that only reflects infrared light sent by the camera. The IR deck is also used to track the RPM of the rotors which, in this case, have some reflective markers attached underneath the blades. Our flight controllers output how much power (PWM) is sent to the individual motors, this information can be used to calculate the residual forces, but knowing the exact RPM values of the blades might be more reliable for this purpose, as there is no delay in

the signal and there is potentially less noise in the data.

2.5 Machine Learning

In this thesis, machine learning was used to try to predict the residual forces acting on the quadrotor, specifically supervised learning to learn a mapping from states to residual forces. The basis for supervised learning is having a dataset of feature and label pairs. Features are what we give to the learning models as input and labels are what we expect the model to output for the corresponding labels. Labels can be continuous or discrete. In the case of continuous labels regression models are used, for example a model that tries to predict house prices given features such as size, location or build year. While with discrete labels classification models [12] are used, an example for this would be a learning model that tries to classify images into ones that contain a cat and ones that don't.

Following are two different learning model architectures, neural networks, specifically multi-layer perceptrons (MLPs), and decision tree ensembles (DTEs). Each of which have their own advantages and disadvantages.

2.5.1 Neural Networks

We use a neural network to make predictions about the residual forces in a given state, specifically a Multi Layer Perceptron (MLP) [24] [5] [13]. An MLP is a universal function approximator which executes alternating linear and non-linear operations on an input vector. If we define an error function that tells us how far off the MLP is from our desired output, we can try to minimize this error through optimization with respect to the MLP's weights.

As mentioned previously, an MLP is comprised of a set of alternating linear and non-linear functions. Each linear operation is followed by a non-linear activation function and is called a layer in the neural network. In the linear operation we find the so-called weights and biases as represented by a matrix and a vector respectively as shown in Equation 11, $\mathbf{b}$ is the bias vector, with dimensions matching the desired output, $\mathbf{W}$ is the weight matrix, with columns equal to the input dimension and rows equal to the output dimension, and $\mathbf{x}$ is the input vector.

$$l(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (11)$$

The weights and biases are the components that we optimize in order to get our desired outputs. The non-linear function component (or activation function) stays fixed and gets defined together with the model architecture. There is a large amount of activation functions to choose from, one of the most popular ones, and the one that we use, is ReLU (Rectified Linear Unit) [19], which is defined as shown in Equation 12 for each element x of the output vector of the linear function.

$$ReLU(x) = \max(0, x) \quad (12)$$

The cost function that is used in this thesis to optimize the model parameters is the mean squared error function defined as follows:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (13)$$

Where y_i is the real, ground-truth label associated with the input features for the data, $\hat{y}_i$ is the prediction made by the model and n is the amount of individual feature-label-pairs in our dataset.

The cost function outputs a scalar value indicating how big the error in our prediction is.

In the most basic case, we aim to minimize the error by calculating the gradient of the loss function with respect to the neural network’s weights and biases. This process, known as backpropagation [24], relies on the chain rule to compute the gradients as they are propagated backward through the network, layer by layer. This allows for the adjustment of weights and biases using gradient descent. Gradient descent can be described by the following formula:

$$\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \nabla J(\boldsymbol{\theta}) \quad (14)$$

Where $\boldsymbol{\theta}$ represents the weights and biases of the MLP, $\nabla J(\boldsymbol{\theta})$ is the gradient of the cost function $J(\boldsymbol{\theta})$ and α , often called learning rate, is a scalar that determines how large each step in the update of the parameters $\boldsymbol{\theta}$ is. In this thesis, the Adam optimizer [11] was used to update the weights of our network which combines gradient descent with momentum.

One property of MLPs is the fact that they can be easily parallelized, this means that using specialized hardware like graphics cards can lead to very fast computations. However, neural networks generally take longer to execute when compared to decision trees as shown in [25].

2.5.2 Decision Tree Ensemble

Inference on a single decision tree [23] is very straight-forward and can be used for tasks such as classification [16]. A decision tree consists of nodes connected by branches. We start inference at the root node, each node contains the index of a value at the input vector and a split parameter. This means that we take a value from the input vector, check if it is less than or equal to a certain parameter stored in the node and move to one of two new nodes depending on the result, these trees are therefore binary trees, as each node that is not a leaf node has two child nodes. If we continue going down the tree, we will eventually land on a leaf node which contains a value indicating the tree’s prediction for a single output value in the output vector. As one decision tree only outputs a single scalar, we need an individual decision tree for each value in the desired output vector.

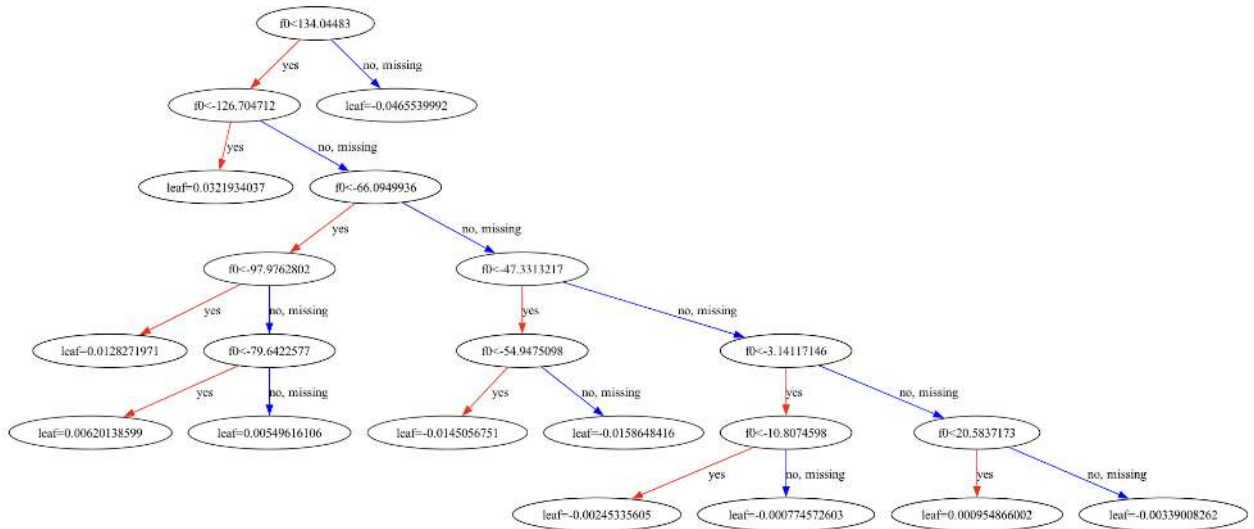


Figure 4: Example of a decision tree for a regression task generated with XGBoost [4]

For regression tasks, it is usually not enough to have a single tree to predict an output, therefore we use a decision tree ensemble (DTE) [2]. This means that we train multiple trees for inference and then take the average over all of them. So if we have a task where we are predicting a six dimensional vector and for each dimension we have five decision trees, our final DTE will have a total of 30 decision trees.

Decision trees allow for an easier analysis of the model’s inner workings [32]. It also makes it easier to check which input parameters are most important for the model inference.

Decision trees can grow to very large sizes, so therefore it is good to set a maximum depth limit to how much they can grow.

2.6 Dynamics Learning

To enhance control in robotic systems, it is essential to develop models that are robust to complex states. Traditional methods for modeling dynamics can be overly complex and may not adequately capture the intricacies of real-world systems. This necessitates the exploration of learning-based prediction methods that can effectively model complex dynamics, even with limited state information.

Learning state dynamics can be approached in various ways. It’s crucial to define the problem clearly and specify what aspects of the system we aim to model. If the physical model of the environment is known, one can learn its physical parameters. However, traditional models may not fully capture the system’s dynamics; they might represent only certain aspects. In cases where the dynamics are entirely unknown, a full dynamics model can be learned from data.

2.6.1 Parameter Estimation

One example of dynamics learning is parameter estimation [8]. In this scenario we know the dynamics model but lack information about our robot’s or the environment’s physical properties, for example, mass, inertia matrices or friction. One can formulate this problem as minimizing the cost function

$$\min_{\theta} \sum_t \|\mathbf{x}_t - f_{\theta}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})\|^2 \quad (15)$$

Where θ represents the physical parameters being estimated, f_{θ} represents the known dynamics functions, t is the timestamp, $\mathbf{x}$ is the current state and $\mathbf{u}$ is the control input. In this problem setting, we assume that we know the dynamics functions.

2.6.2 Full Regression

Another approach would be to try to model the full dynamics model of the robotics system [9] [6]. This means that for a given state $\mathbf{x}_{t-1}$ action $\mathbf{u}_{t-1}$ pair, the model has to learn to predict the next state $\mathbf{x}_t$ without any prior knowledge of the system, learning the full dynamics of the system rather than some parameters in a predefined dynamics function. Same formulation as 15 but in this case θ represents the weights of some learning model f_{θ} . This problem formulation is more complex as we try to learn the full dynamics of the system and is mainly useful in scenarios where we know very little of the dynamics model, but have access to a large amount of data.

2.6.3 Residual Dynamics

An alternative approach, which will be utilized in this thesis, involves learning the residual dynamics of our existing dynamics model [1] [28] [25]. In this scenario, there is an approximate dynamics model available that captures the majority of the system’s behavior but fails to account for certain complex forces that are challenging to model accurately. To address this limitation, we can apply a learning-based method to adjust the model.

This problem can be formulated as:

$$\min_{\boldsymbol{\theta}} \sum_t \|\mathbf{x}_t - [f_M(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) + f_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})]\|^2 \quad (16)$$

Here, f_M represents the known physics-based model of the system, while $f_{\boldsymbol{\theta}}$ is the approximate function to model the residual components parameterized by $\boldsymbol{\theta}$, $\mathbf{u}_t$ and $\mathbf{x}_t$ are the same as before. By focusing on learning only the residual dynamics rather than the entire dynamics model, we can achieve greater data efficiency and reliability in our predictions.

2.6.4 Observation-based Dynamics

In some cases, we may not have complete state information at each timestamp, or the environment might not exhibit Markovian properties, this means the transition from the current state to the next is not affected by any previous states. In such scenarios, it is beneficial to utilize sequence models [10] [31], that can take a series of observations, which may only contain some information about the state, to make accurate assumptions about the full state of the environment. For instance, a model that predicts the velocity of a robot based on its position at each timestamp would require information about its previous position as well [29] [26].

Various model types can be employed to address this issue. Autoregression, for example, utilizes a sliding window approach, where the model’s input consists of a concatenation of all previous states over the last h timestamps:

$$\mathbf{y}_t = f_{\boldsymbol{\theta}}(\mathbf{y}_{t-h:t-1}, \mathbf{u}_{t-h:t-1}) \quad (17)$$

Here, $f_{\boldsymbol{\theta}}$ represents the dynamics function parameterized by $\boldsymbol{\theta}$. The term $\mathbf{u}_{t-h:t-1}$ denotes the control inputs from the previous timestamps, $\mathbf{y}_{t-h:t-1}$ indicates the prior observations, and $\mathbf{y}_t$ is the expected observation at the current timestamp. Depending on the specific problem, one might select a larger or smaller value for h , often referred to as the history.

3 Related Work

A lot of literature focuses on learning the residual forces acting on drones or other types of robots. In comparison to learning the full control signal to be sent to the robot, learning just the residual forces is a lot more straightforward. As mentioned previously 2.3, there is a wide range of controllers for quadrotors which already model most of the forces acting on the robot, therefore making use of them, and then adding the non-modeled residual forces, is the most common approach to improving flight performance through learning.

The paper by Bauersfeld et al. [1] proposes an architecture that aims to model aerodynamic forces and torques. This gets added into a standard first principle model to adjust the forces and achieve more accurate control of a quadrotor. This approach is similar to the one used in this thesis, but there are some key distinctions. In the paper, they make a non-markovian assumption. This means the model input is defined by a sliding window which takes a sequence of previous states and the current state as input, while in this thesis, we only use the current state as input to our learning models.

Another important point to make is the much larger scale of the data and the type of quadrotor being used. The data was collected in one of the largest motion capture facilities in the world, with the quadrotors reaching speeds of up to 65 km/h. The resulting dataset contained 96 flights amounting to a total flight time of 1h 15min or around 1,800,000 timestamps. While our approach takes data from a much smaller motion capture area, with the limited Crazyflie hardware and under 200,000 timestamps.

This paper uses a different approach to calculate the residual forces as can be seen in 18, where $\mathbf{q} = [q_w \ q_x \ q_y \ q_z]^T$ represents the rotation quaternion of the quadrotor, $\cdot$ represents the rotation of a vector via a quaternion and $\mathbf{g} = [0 \ 0 \ -g]^T$. The force signal outputted by the flight controller is represented as $\mathbf{f}$ and $\mathbf{f}_a$ is the residual force. Same for the $\boldsymbol{\tau}$ and $\boldsymbol{\tau}_a$, but in this case for the torque. The difference here for how the residual forces get calculated is that they are in the quadrotor frame rather than in the world frame.

$$\begin{bmatrix} \dot{\mathbf{r}} \\ \dot{\mathbf{q}} \\ \dot{\mathbf{v}} \\ \dot{\boldsymbol{\omega}} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \dot{\mathbf{q}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q} \odot (\mathbf{f} + \mathbf{f}_a)) + \mathbf{g} \\ \mathbf{J}^{-1}(\boldsymbol{\tau} + \boldsymbol{\tau}_a - \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega}) \end{bmatrix} \quad (18)$$

It is not quite clear how the residual forces were computed in the code implementation as this is not present in the source code, and the already calculated residual forces are in the dataset for training the learning models.

The paper by Shi et al. [28] focuses on the interaction forces between multiple quadrotors rather than a single one like in [1] or this thesis. It uses deep sets to combine information from a random number of quadrotors. One of the main concerns in this paper is the so-called downwash effect, which occurs when one quadrotor flies over another, causing the lower one to be pushed downwards. We don't experience this type of effects in this thesis as there is only one quadrotor flying at a time. In the paper, the forces acting on the x - and y -axis are neglected, as the main residual force in the types of scenarios presented mostly reflect on the z -axis, making the residual forces in the x - and y -axis negligible for this context. The method described in this paper managed to reduce the worst-case z -error by a factor of two to four. Very similar hardware was used to the one used

in this thesis.

The thesis by Schicke [25] focuses on the development of two different types of learning model architectures, for the prediction of residual forces on a Crazyflie quadrotor. The thesis showed that it's possible to make accurate predictions about the residual forces acting on quadrotor. The main focus was on the development of learning model architectures that could achieve high accuracy in predicting the residual forces, but they were not tested on real world hardware to see if they had any impact on flight performance.

In the thesis it was shown that decision tree ensembles provide a better accuracy and faster inference/training speed for a quadrotor residual force prediction task. However the size of decision tree ensembles was not explored, which is a big factor when trying to the generated tree on limited hardware. Decision trees can grow to very large sizes. As we will see later 4.4.2, this is a big issue in our case.

4 Methodology

The prediction models that we run onboard of the Crazyflie were originally developed in a previous thesis by Schicke [25]. In said thesis the focus was on creating models that can approximate the residual forces, while in this thesis, we run the models on-board the actual hardware. The idea is to create a pipeline for transforming the models into code executable by the Crazyflie hardware and then testing the accuracy, inference time and overall flight improvement caused by these models. In the following, the methodology is further described as well as some experiments that were done to test the correctness of the residuals we calculated for training the models.

In our case, it is not really necessary to estimate the residuals since we have access to all the information we need to compute these values, however, predicting the residual forces with learning models might smooth out the residual force values, which can be noisy when computed with state-of-the-art methods. In addition, creating a pipeline for a system that can transform models from Python code into C code will allow for easy deployment on the hardware for future scenarios where information might be limited.

4.1 Final Pipeline

Ideally, we want to end up with a system that allows for a simple transfer from the initial training data and model definition to an updated Crazyflie firmware. For this purpose, code was developed that implements the pipeline depicted in Figure 5.

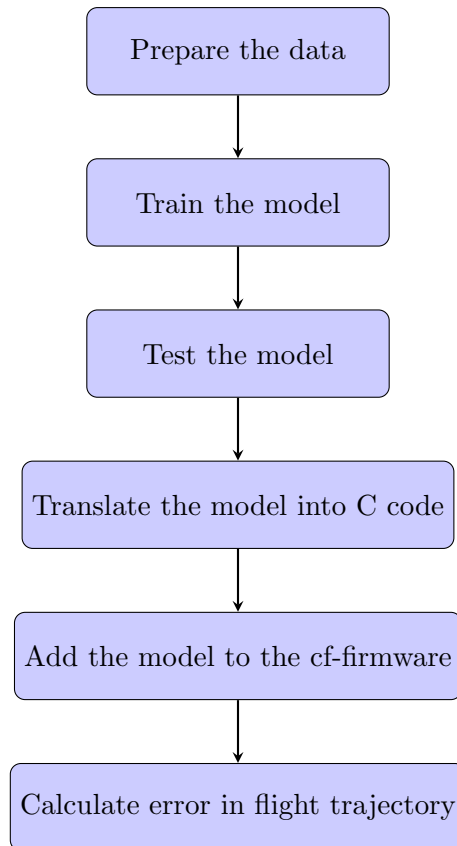


Figure 5: Learning model to C-code pipeline

In the first step, we load flight data and calculate the corresponding residual values depending on the quadrotor hardware. We split the relevant information into training and testing sets. After that, we define a model, either a decision tree ensemble or multi-layer perceptron, and train it. Once the model is trained we can test it to see if its predictions are correct. Once the model is ready, it can be transferred into C code. The newly generated C code gets compiled and fed some random input data from the testing set, the same data gets fed into the original model and then the outputs are compared. This sanity check is only relevant when updating the C code generation. Then the new code gets automatically added to the Crazyflie firmware. The last step involves doing some flights to collect flight trajectories on which we can measure the deviation from the desired flight path.

4.2 Initial Experiments

In order to check if the residual forces ⁷ that we are calculating are reasonable, we first tried plotting them on a straight flight path as can be seen in Figure 6. The plot shows a zoomed in section of a straight path towards the negative y -axis direction. The blue line represents the desired path to be followed while the orange line shows the real trajectory taken by the quadrotor. As we would expect the residual forces point in the opposite direction to the quadrotor’s velocity assuming the velocity is pointing towards the desired path. This tells us that part of the residual force being modelled is the drag, which opposes motion and increases with speed. The plot also shows some of the initial predictions for the residual forces from a neural network, further discussed in section 4.3. The hope is therefore that by subtracting the predicted residual forces from our control input,

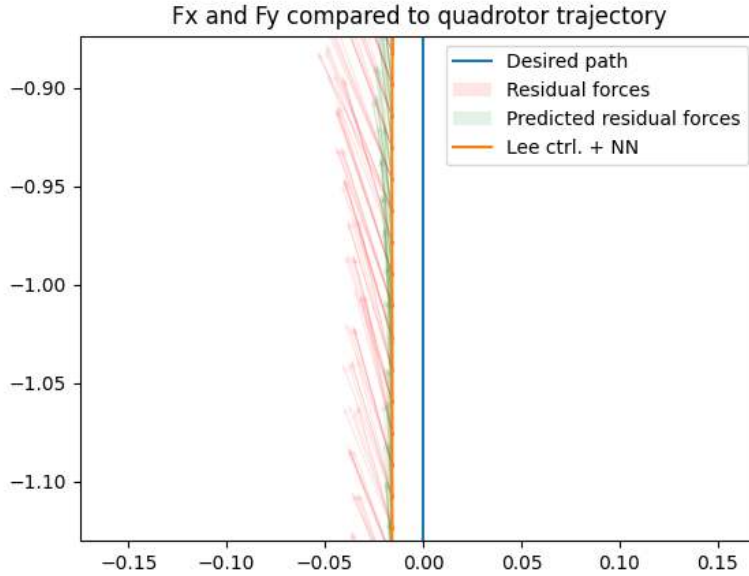


Figure 6: The residual forces acting on a quadrotor that is moving on a straight path towards the negative y -axis direction. As can be seen the residual forces, in red, push the quadrotor in the opposite direction of its target path, represented by the blue line. The green arrows show the predictions of a neural network and the orange line is the real trajectory taken by the quadrotor.

the Crazyflie should get closer to the desired path during flight [1].

The higher the velocity of a quadrotor, the higher the residual forces acting on it will be [15] [7]. The residual forces should also tend to point in the opposite direction of the velocity. To prove this and to check the correctness of our residual calculations, the x and y values for the velocity and residual forces were plotted for each timestamp in a path taken by the quadrotor, the results are shown in Figure 7.

As can be seen from the plot our assumptions about the nature of the residuals seems correct.

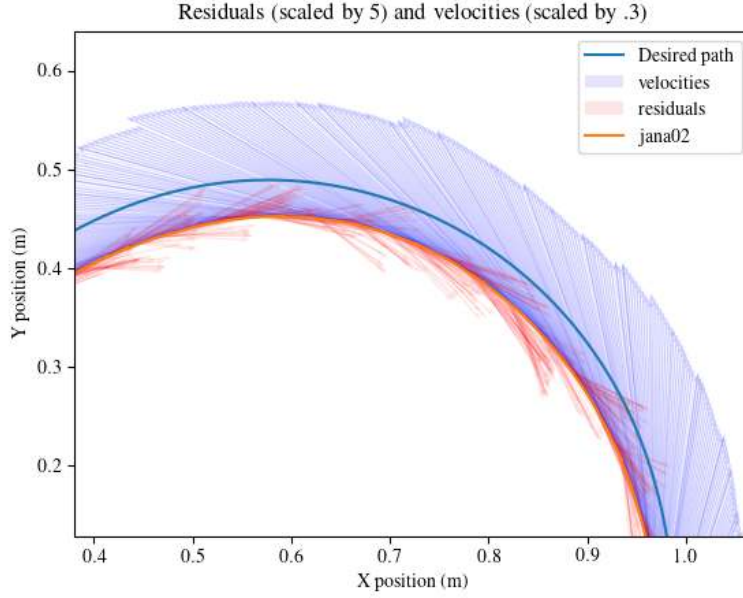


Figure 7: xy plane view of the residual forces (red, scaled by 5.0) and velocities (blue, scaled by 0.3) on a flight path. There is a clear correlation between the velocity of the quadrotor and the residual forces on it. The blue line represents the desired trajectory and the orange line represents the real path taken by the quadrotor.

Similarly, for higher thrust we should expect higher residual values, since the drone will be fighting against aerodynamic forces like air resistance. To visualize this, I calculated the norm of the x and y components of the current thrust as well as the residual forces during a flight. The results can be seen in Figure 8. Once again we can see that our assumption was correct and that a higher thrust causes higher residual values.

In order to calculate the residual forces we need the total thrust of the Crazyflie. To get this value we have two options, we can use the power sent (PWM) to each rotor, or each rotor's current speed in RPM. To get the translation from PWM or RPM to thrust we can simply fit a second-order polynomial to some recorded values. The polynomial coefficients used for this were taken from the Crazyflie-data-collection GitHub repository of the IMRC Lab ¹. To collect RPM data, we used the IR deck mentioned in the experimental setup section 2.4. As the RPM data is recorded directly from the rotors, we might assume that taking the force calculated through RPM values may have less delay. To prove this, we plot the experimental results. What we should expect to see is that the residual values calculated through PWM are shifted to the right with respect to the ones calculated

¹GitHub repository: <https://github.com/IMRCLab/Crazyflie-data-collection>

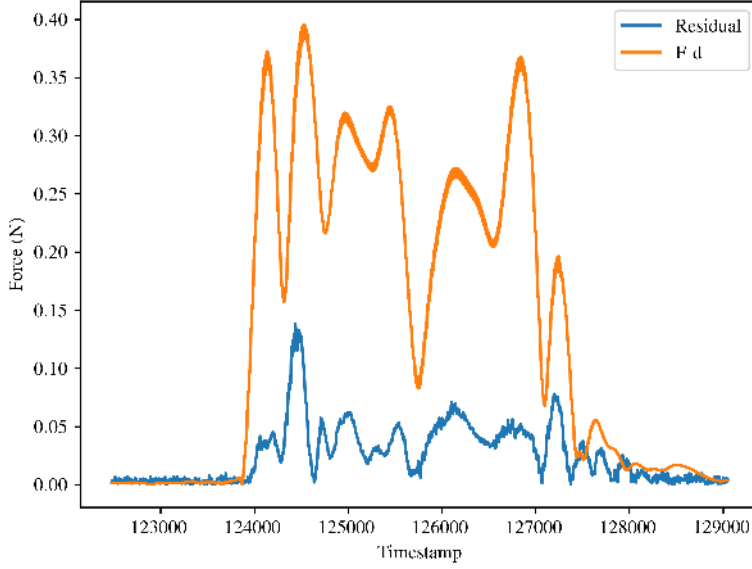


Figure 8: Comparison of the residual forces (blue) in relation to the total thrust outputted by the flight controller (orange). In this plot only the forces in the x and y directions were taken into account, that means, the length of the vectors in two dimensional space.

with RPM. The results can be seen in Figure 9. The results show that there is slight delay for the PWM, although for the most part it's negligible. We use the residual forces calculated from the RPM to train our models as this will yield slightly more accurate results.

Another notable observation was seeing how the total residual forces in x and y evolved over time on a figure-eight flight path (see Figure 12). The results of this experiment are displayed in Figure 10. The largest forces appear at the beginning of the flight, they can be divided into three distinct peaks that reduce in size over time. One explanation for this decrease in the total magnitude could be the controller adjusting to the flight conditions.

4.3 Estimating the Residuals with Learning Algorithms

The models that we base this work on by J. Schicke [25] try to predict the full six dimensions of the residuals forces, however we only apply the x and y components of the translational forces $\mathbf{f}_a$ on the controller, since these are the most relevant in our single quadrotor scenario.

A wide variety of different combinations of the drone state and measurements were tested as input for the learning models to find the ones that lead to the best performance. In the end, the models performed best by taking a 12-dimensional vector as input with the acceleration in x , y and z direction in the quadrotor coordinate system, the gyroscope x , y , z measurements, and the first two column vectors of the rotation matrix of the quadrotor. In the original thesis that proposed these models the acceleration was multiplied by 9.81 and the gyroscope measurements were in radians.

As mentioned previously, the output of the models in [25] is a 6 dimensional vector of which the

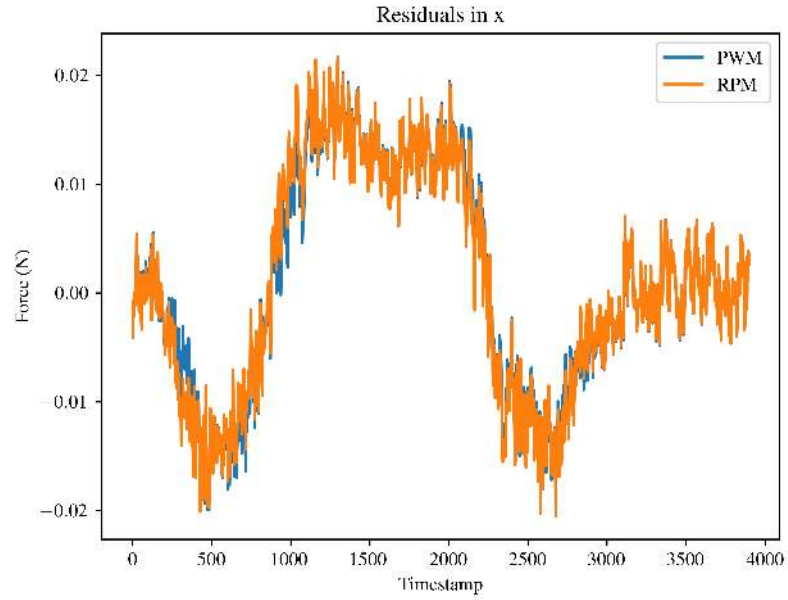


Figure 9: Comparison between the residual in x calculated by the PWM(blue) and the one calculated by RPM(orange)

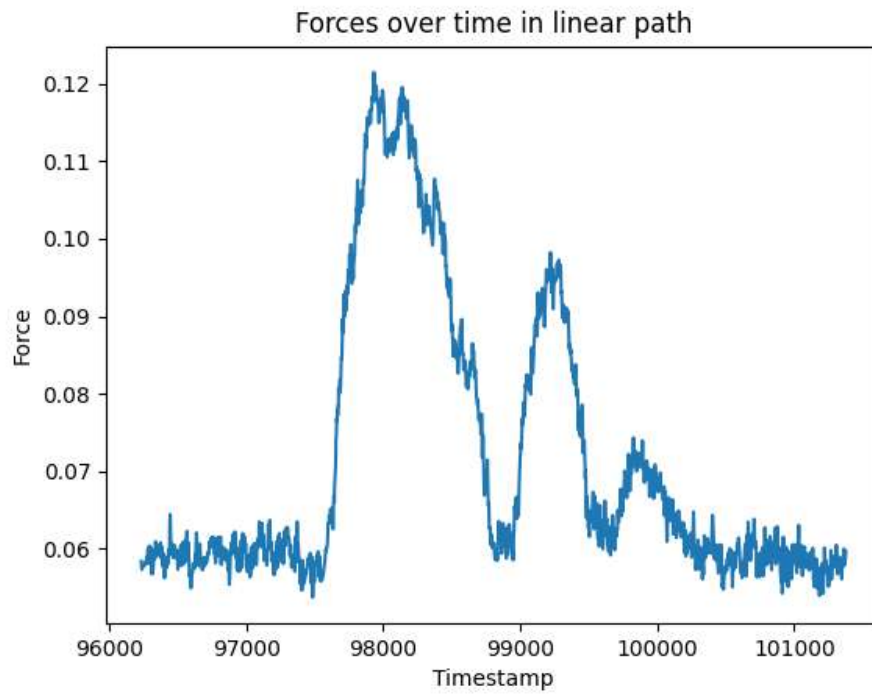


Figure 10: Residual forces over time on the figure-eight flight path

first three values represent the force residual, and the next three the torque residual. For a better training process, the residuals also get scaled, as otherwise they are too small for the model to be able to learn efficiently. This is done by taking the maximum and minimum values for each of the training data label values and scaling them to be values between -1 and 1 with min-max-scaling as shown in 19 where x' is the updated value of x , $x_{\min}$ is the smallest possible value that x can have and $x_{\max}$ is the largest possible value.

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (19)$$

For this thesis, we are only interested in predicting the x and y components of the residual calculation as those are the only ones that were relevant in our case. In addition, predicting the torques and z forces has proven to be more difficult than the residual forces in x and y , which would only reduce performance in flight if applied to the flight controller. The input variables remained the same but the scaling step of the acceleration and gyroscope measurements were omitted. The original decision tree ensembles were made with XGBoost [4], but due to some complications with reading the model parameters, we switched to Scikit-learn [22] for the DTEs, which we found to have an easier interface for reading the internal parameters of the individual decision trees. While Scikit-learn is slower at training DTEs, the structure remains the same, so therefore the inference time will also stay the same when transformed into C code.

As can be seen in Figure 11, the approximation by our learning models, based on Schicke [25], is quite accurate.

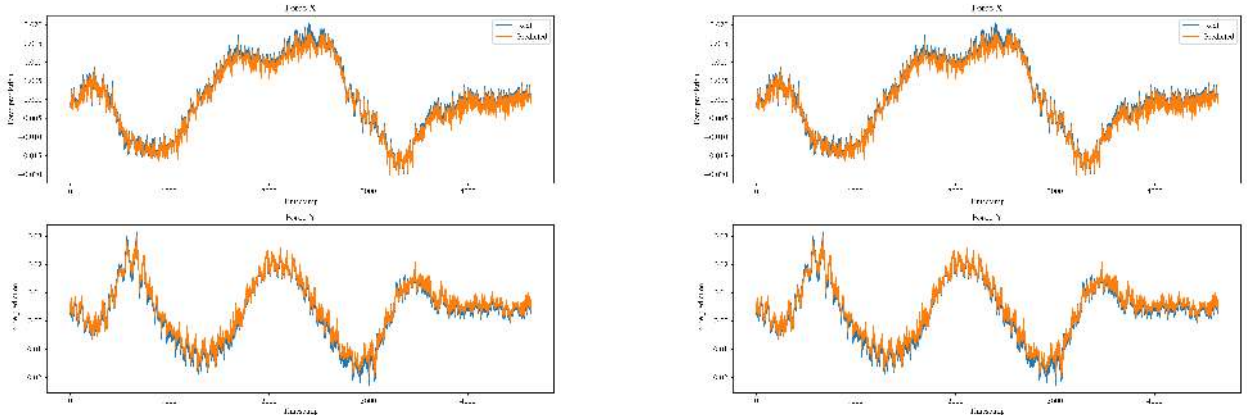


Figure 11: Off-line predictions for the x and y forces from the neural network(left) and the decision tree ensemble(right). The average error for the MLP is 0.00185 and for the DTE 0.00135

4.3.1 Initial Model Comparison

After training our new models we compared the accuracy on each of the two model architectures. Different configurations have been tested for both architectures to optimize for inference time and accuracy [25], so we only adjusted the training data while maintaining the same overall model structure. Table 1 shows the results for the standard Crazyflie, while Table 2 shows the results on the brushless Crazyflie, note that the models trained for the brushless Crazyflie were trained on 30% more data. It is clear that the decision tree ensemble outperforms the neural network,

although it has a much larger amount of parameters which leaves little space for extra code in the STM32.

Table 1: Off-line model accuracy comparison for standard Crazyflie

	avg. error in f_x	avg. error in f_y	approx. param. #
Decision tree	0.0012 N	0.0015 N	13,500
MLP	0.0018 N	0.0019 N	798

Table 2: Off-line model accuracy comparison for brushless Crazyflie

	avg. error in f_x	avg. error in f_y	approx. param. #
Decision tree	0.0006 N	0.0007 N	13,500
MLP	0.0004 N	0.0007 N	798

The new MLP takes the same 12 dimensional input as the model proposed by J. Schicke but only outputs a 2D vector. It has two hidden layers of size 16 and uses the ReLU activation function on all layers except for the output layer.

Our decision tree ensemble shares the same input and output dimensions. It uses 5 estimators per output dimension, so 10 individual regression trees, where each tree has a maximum depth of 10 nodes which leads to around 300 non-leaf nodes and 300 leaf nodes per tree.

4.4 Translating the Models Into C Code

As previously mentioned, the Crazyflie operates using an STM32 microcontroller, which lacks the flexibility of the Python environment originally used to develop the models. As a result, uploading these models to the quadrotor requires a thorough understanding and analysis of their structure to accurately read and reconstruct them within the microcontroller’s constraints.

In order to be able to run our models on this restrictive hardware, we have to find a way to transport the model parameters into C code. Which is a language that runs very close to the hardware and is used to write code on most microcontroller such as the STM32. We have to write two different python scripts to translate the neural networks from PyTorch [21] and decision tree ensembles from Scikit-learn [22] into compilable C code. Writing python code to generate the corresponding C code will also allow for a convenient way to transform future models into the necessary format for the Crazyflie.

Bitcraze provides the Crazyflie with an open source firmware². When implementing the models into the firmware, they need to be able to read the state of the quadrotor and adjust the output of the controller with the predicted residual. To do this we need to take a look inside of the controller code, which is responsible for reading the current state of the drone and adjusting the desired forces on each rotor with respect to the desired flight path. We apply the residual calculations on the position controller of the Lee controller. To do this we modify equation 9:

$$\mathbf{f} = -(-k_{\mathbf{r}}\mathbf{e}_{\mathbf{r}} - k_{\mathbf{v}}\mathbf{e}_{\mathbf{v}} - m\mathbf{g}\mathbf{e}_3 + m\ddot{\mathbf{r}}_d - \mathbf{f}_a) \cdot \mathbf{R}\mathbf{e}_3 \quad (20)$$

²GitHub repository: <https://github.com/bitcraze/crazyflie-firmware>

Now the total force $\mathbf{f}$ outputted by the controller gets adjusted by our predicted residual force $\mathbf{f}_a$. The MLP and DTE will run in the same control loop as the Lee controller. Inside of the controller's step function we find all relevant state information necessary as input to our models, as this same information needed by the Lee controller itself. The Lee controller calculates a total force $\mathbf{f}$ that is then sent to the quadrotor's motors. This computed force is a 3D vector that defines the forces that need to be applied on each of the xyz -axis, so all we need to do is gather the input data for our network, transform it as we did when creating our training data, and then apply the $\mathbf{f}_a$ outputs to the $\mathbf{f}$ after propagating the information through the model.

After each model gets transformed into C code it is important to test if the outputs are the same as the original model for a given set of inputs. This is done automatically within the same C code generation script.

4.4.1 Neural Network

For the neural network we can divide the C code into some utility functions, that will stay the same for every model, and scripts that store model specific information like weights and biases as well as the overall network structure.

In the *utils_nn.h* and *utils_nn.c* scripts you will find the layer function which takes the values of some previous layer, the weights and biases of the next layer and propagates the information as can be seen in Algorithm 1. The layer propagation function will be the same for all neural networks and therefore can be written directly and doesn't need to be generated with a python script. This propagation through the network is done via for loops since, with the standard STM32, we can't parallelise the calculations like how PyTorch [21] or other libraries for training models would do it making use of specialized hardware such as graphics cards.

Algorithm 1 Layer Forward Pass

```

1: procedure LAYER(rows, cols, input[rows], layer_weight[rows][cols], layer_bias[cols], out-
   put[cols], use_activation)
2:   for  $i = 0$  to  $cols - 1$  do
3:      $output[i] \leftarrow 0$ 
4:     for  $j = 0$  to  $rows - 1$  do
5:        $output[i] \leftarrow output[i] + input[j] \times layer\_weight[j][i]$ 
6:      $output[i] \leftarrow output[i] + layer\_bias[i]$ 
7:     if  $use\_activation == 1$  then
8:        $output[i] \leftarrow \text{ReLU}(output[i])$ 

```

Now we can take a look at the *nn.h* and *nn.c* scripts. These have been generated by a python script which takes a .pth file and reads it's contents. The .pth files contain a structure which defines the weights and biases of a neural network. They also give us the necessary information to define a forward function which propagates a given input through the network making use of the previously defined single layer propagation functions in *utils_nn.h* and *utils_nn.c*.

4.4.2 Decision Tree Ensemble

For the decision tree ensemble we have a similar system where the *tree_utils.h* and *tree_utils.c* files define a basic tree transversal function and the files *tree.h* and *tree.c* are generated by the node

values of the tree stored in the Scikit-learn [22] decision tree ensemble file. In the *tree.h* and *tree.c* we define a *tree_forward* function which calculates the mean over all the trees in the ensemble for each output class.

The standard way of storing decision trees in libraries like Scikit-learn [22] or XGBoost [4] however uses a lot of storage space, as for each node on a tree five parameters are stored; the left child, right child, split index, split condition and output value. Since each individual tree has around 50% leaf nodes, half of the nodes have 4/5 values which simply don't get used for inference, while the rest of the nodes also have 1/5 values which don't get used. This is fine in a regular computer which has more storage and memory, but infeasible on the Crazyflie hardware. So therefore we change our tree structure so that the leaf nodes and parent nodes are stored in separate arrays. Since there is a total of 12 input values the split indices can be stored in 8 bit integer values, while all values referring to other nodes like left/right child, can be stored in 16 bit values as there are never more than 1000 nodes in the trees we created for this task, given the max tree depth of 10. To indicate if a given child node is a leaf node we can set any of the final bits in the integer referencing the leaf node to a 1 and then check if this bit is set to 1 each time before moving to a new child node in the forward propagation function of a decision tree, as can be seen in the new tree transversal Algorithm 2.

Changing the integer sizes in our decision trees and modifying how we represent the individual nodes, reduces the total size of the final DTE by about 76%, however, using the DTE leaves little space left for extra code in the STM32.

Algorithm 2 Decision Tree Inference Algorithm

```

1: function DECISION_TREE_INFERENCE(left_children, right_children, split_indices,
   split_conditions, output_value, input)
2:   index  $\leftarrow$  0
3:   while  $\neg(\text{index} \& 0b0100000000000000)$  do
4:     if split_conditions[index]  $\geq$  input[split_indices[index]] then
5:       index  $\leftarrow$  left_children[index]
6:     else
7:       index  $\leftarrow$  right_children[index]
8:   return output_value[index  $\&$  0b1011111111111111]

```

4.4.3 Standard Residual Calculation

As we have access to all the current state information it is also possible to do the residual calculation directly, leaving out the learning and extra c code generation. However, learning methods will tend to generate smoother functions, taking the direct residual calculation approach might give us more noisy predictions. Nevertheless it will be interesting to see what results we get from using this method to predict the residual forces.

Calculating the residual forces in C code is very straightforward as we only need the translational x and y values of said forces. There is no need for extra libraries as the calculations can be simplified a lot. Algorithm 3 shows how we calculated the residuals directly onboard the Crazyflie, where $R02$ and $R12$ are elements of the rotation matrix and w_{acc_x} and w_{acc_y} are the x and y values of the acceleration vector in world coordinates. In this pseudocode the residual is calculated with the PWM values, but it could be adapted to use RPM instead, by changing the a, b and c values

in the force calculation function.

Algorithm 3 PWM to Force Conversion and Basic Residual Calculation

```

1: function PWM_TO_FORCE(pwm)
2:    $a \leftarrow 1.65049399 \times 10^{-9}$ 
3:    $b \leftarrow 9.44396129 \times 10^{-5}$ 
4:    $c \leftarrow -0.377748052$ 
5:   return  $pwm^2 \times a + pwm \times b + c$ 
6: function BASIC_RESIDUAL(output, pwm, R02, R12, w_acc_x, w_acc_y)
7:    $total\_force \leftarrow 0$ 
8:   for  $i \in \{0, 1, 2, 3\}$  do
9:      $total\_force \leftarrow total\_force + pwm\_to\_force(pwm[i])$ 
10:   $total\_force \leftarrow total\_force \times g2N$ 
11:   $output[0] \leftarrow w\_acc\_x \times QUADROTOR\_MASS - R02 \times total\_force$ 
12:   $output[1] \leftarrow w\_acc\_y \times QUADROTOR\_MASS - R12 \times total\_force$ 

```

4.5 Testing the Performance

First we test how fast the model can perform inference inside of the controller. Ideally we would like to see an inference time of over 60 Hz in order for the models to even be practical. It is expected that the decision trees will have an advantage given previous work [25]. However the implementation of the models on C code differs from the more common python implementations since, as discussed earlier, the Crazyflie hardware does not provide us with the same amount of resources as the hardware used in the training and testing process in the original development phase of the prediction models. Therefore we can not directly assume that the runtime will be proportionally the same as the original measurements. Before we start flying we measure the time between the start and the end of the inference, including the preparation of the input and output vectors for the models (although the latter is probably negligible).

To test the model performance we take some predefined paths for the drone to follow and check how far it diverges from the planned path when trying to follow it. To do this we measure the quadrotor position at each time-step and compare it to the desired position by calculating the euclidean distance as can be seen in Figure 12. We first do this with the default Lee controller in the Crazyflie firmware and then test it with the new models. Since the error can vary between flights we measure it for around 6 separate flights for each method.

What we hopefully find is that adding the models into the controller will make it so that the quadrotor stays closer to the path we want it to follow. The biggest improvement should come from high speed flights.

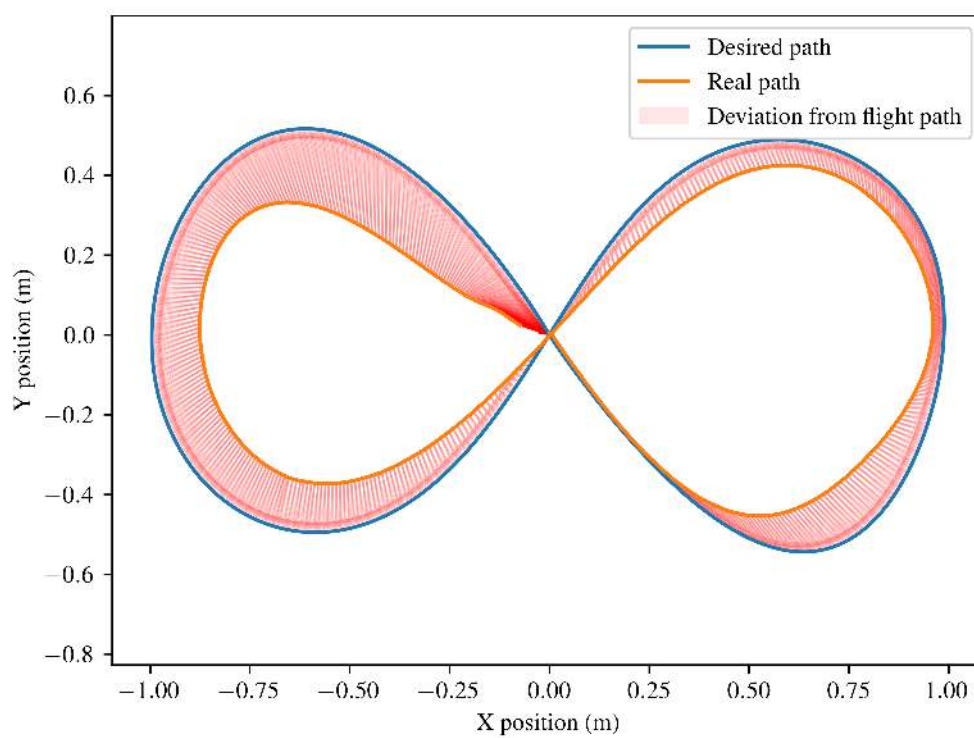


Figure 12: Example of how the error in a flight was measured. At each timestamp the euclidean distance to the desired path is measured, the error is then expressed in meters.

5 Results

To test the impact of the models on the flight controller performance we first use them in a simulation setting after which we test them on real world flights.

For the real world tests, we evaluated each approach on 6 individual flights of the figure-eight flight path on a timescale of 0.7 (avg. velocity: 1.3 m/s) and later 0.6 (avg. velocity: 1.5 m/s). We expect the proportional improvement to be higher in the faster flights as the residual forces will be more prominent.

5.1 Simulation

In order to simulate the residual forces acting on the quadrotor, the neural network that was trained previously was added to the simulation backend in order to map states to residual forces. We expect that the addition of our learning models should have greater impact on simulation than in the real world, as the simulation environment is more predictable and the residual forces are being generated by the same network that we use to compensate for them.

Table 3 compares the deviation from the desired path in simulation for each residual calculation model as well as the standard Lee controller and the simulation with no f_a added. For each category, a single flight was performed on the figure-eight flight path. Since the residual prediction model used to generate simulated residual forces is the MLP, we can see that the MLP added to the Lee controller performs better than the DTE, this does not mean that it will also perform better in the real world.

Table 3: Final results in simulation

	avg. error (m)
No f_a in sim and std. Lee	0.0547
Standard Lee controller	0.0806
MLP	0.0552
Decision tree	0.0607

5.2 Real World Flights

Tables 4 and 5 compare the deviation from the desired path in real world flights for each residual calculation model as well as the standard Lee controller. Figure 18 shows that the estimations made by the decision tree ensemble are quite blocky at the edges when compared to the estimations made by the neural network 15. Overall the error difference between models is relatively small for the standard Crazyflie. There is a noticeable improvement over the basic Lee controller with no residual force prediction.

5.2.1 Standard Lee Controller

As a baseline we did some flights using the Lee controller without the addition of any residual estimation model. The path deviation on one of these flights can be seen in Figure 13.

Table 4: Final results on real flights with the standard Crazyflie (each category averaged over 6 flights on figure-eight with timescale 0.7)

	avg. error (m)	extra computation time (Microseconds)
Standard Lee controller	0.11052	–
MLP	0.08317	56.1001
Decision tree	0.08392	12.4302
Basic calculation	0.0903	2.4026

Table 5: Final results on real flights with the standard Crazyflie (each category averaged over 6 flights on figure-eight with timescale 0.6)

	avg. error (m)	extra computation time (Microseconds)
Standard Lee controller	0.14296	–
MLP	0.1119	56.1001
Decision tree	0.12417	12.4302
Basic calculation	0.1220	2.4026

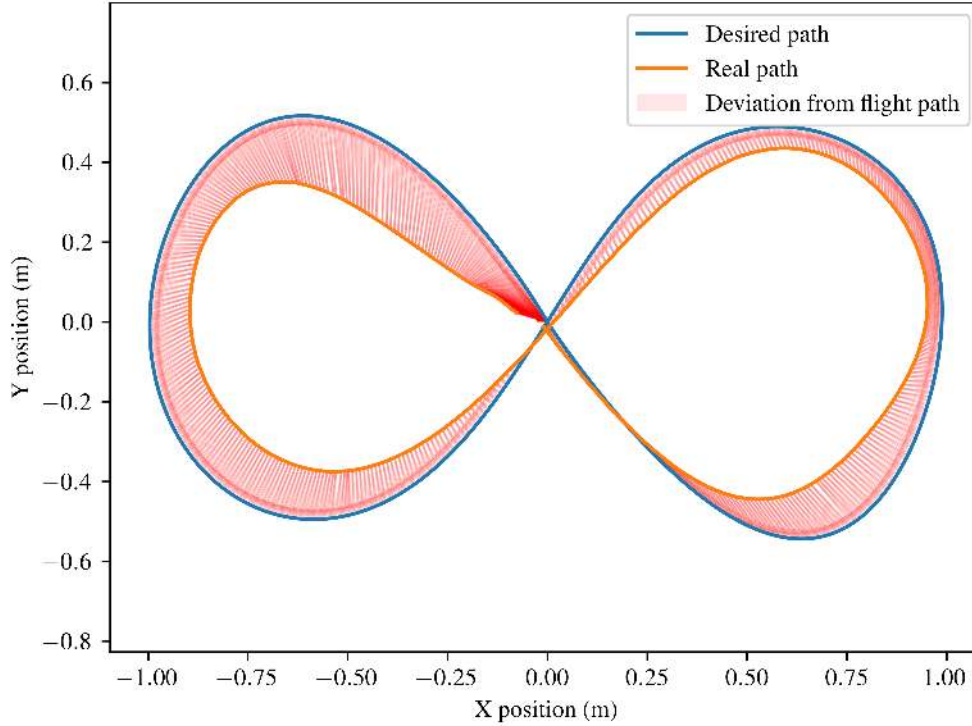


Figure 13: Deviation from the desired path using the Lee controller without any modifications on the standard Crazyflie on a 0.7 timescale flight.

5.2.2 Neural Network

Even without parallelization, the neural network was able to run safely on-board the Crazyflie. As can be seen from the residual calculations in Figure 15, the predictions by the neural network are shifted to the left compared to the residual calculations made after the flight. This might be due to the fact that the residual calculations made post-flight were done with the power signal sent to the motors, while the neural network was trained on the residual calculations made using the RPM values of the blades.

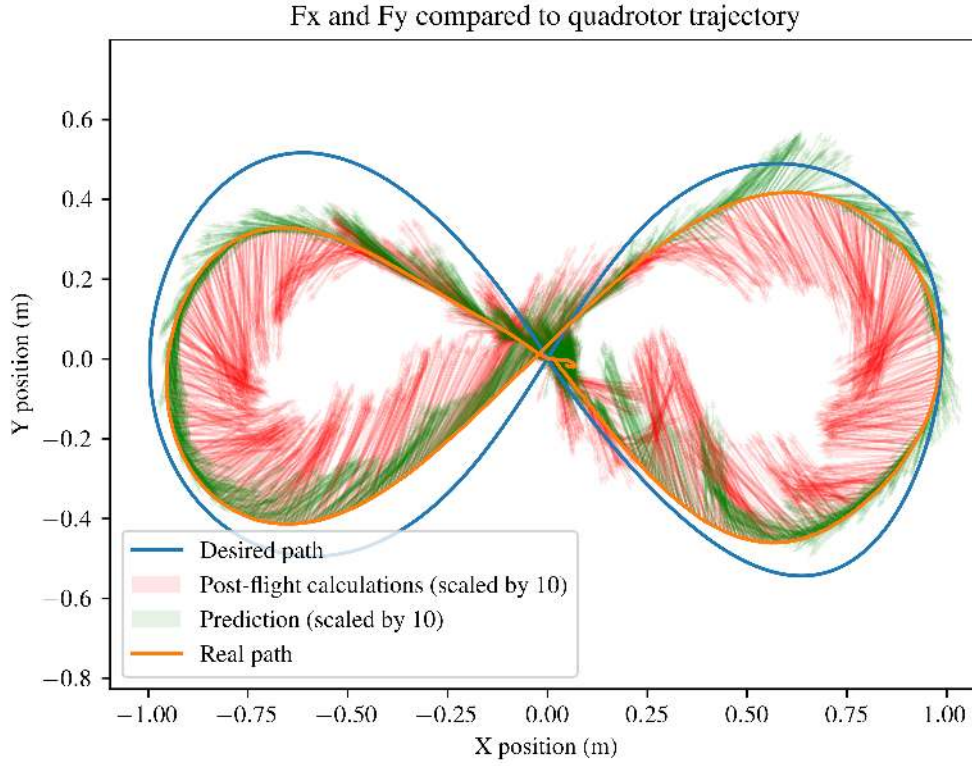


Figure 14: On-flight residual calculation of the Neural Network compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

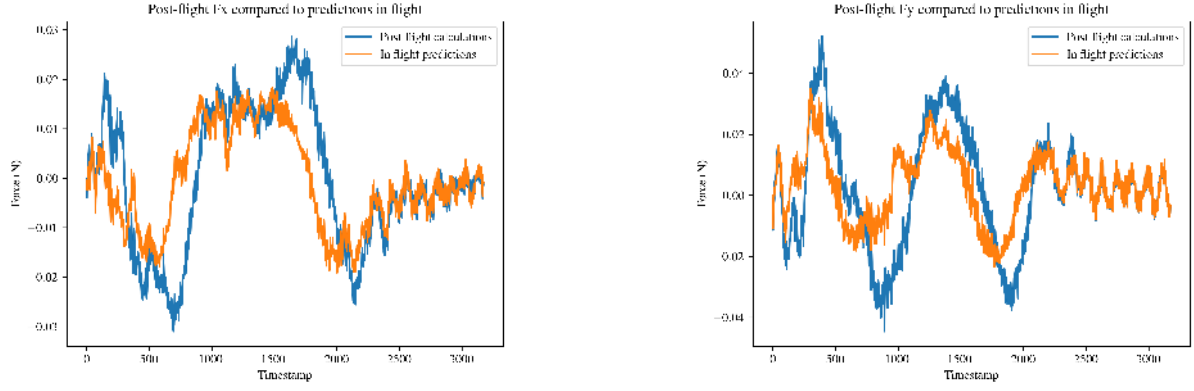


Figure 15: On-flight residual calculation of the Neural Network compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

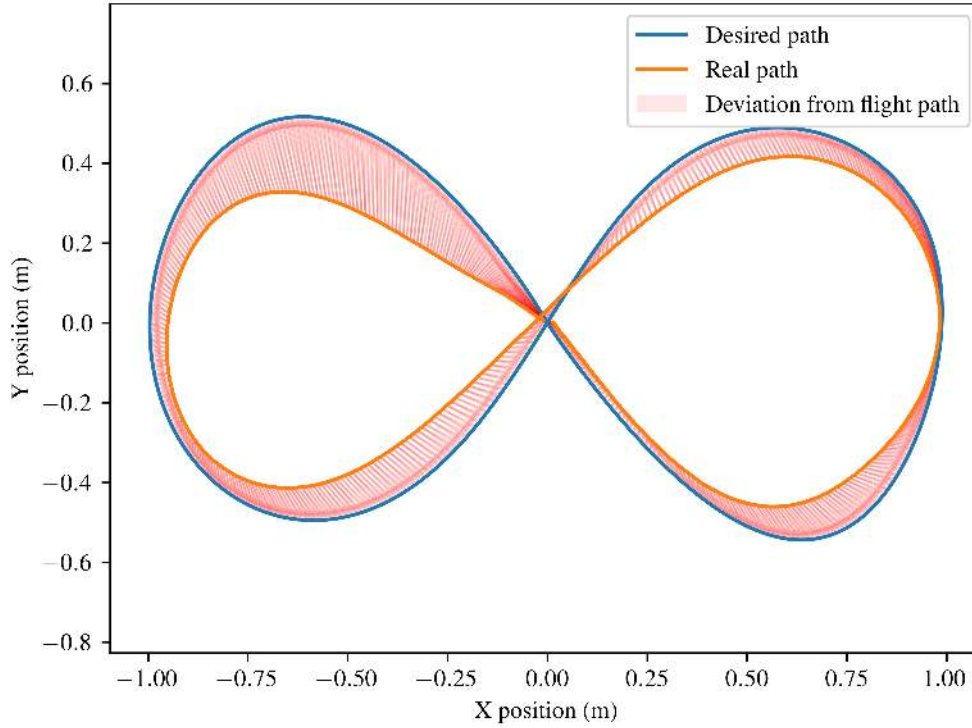


Figure 16: Deviation from the desired path using the Lee controller with the neural network for residual force estimation on the standard Crazyflie on a 0.7 timescale flight.

5.2.3 Decision Tree Ensemble

The decision tree ensemble has a very fast inference time, especially when compared with the neural network. Previous tests [25] showed this but were only done on regular computers which have hardware that can better parallelise the matrix multiplications found in neural networks. Therefore the inference time difference is further emphasised. Once again, like with the neural network, the residual calculations made on-board by the Decision Tree Ensemble, seem to be shifted to the left, as can be seen in Figure 18. The predictions seem to have a blocky shape, this could be fixed by adding more trees to the ensemble, but this is not possible, as the current decision tree ensemble already is close to the maximum size possible on the STM32.

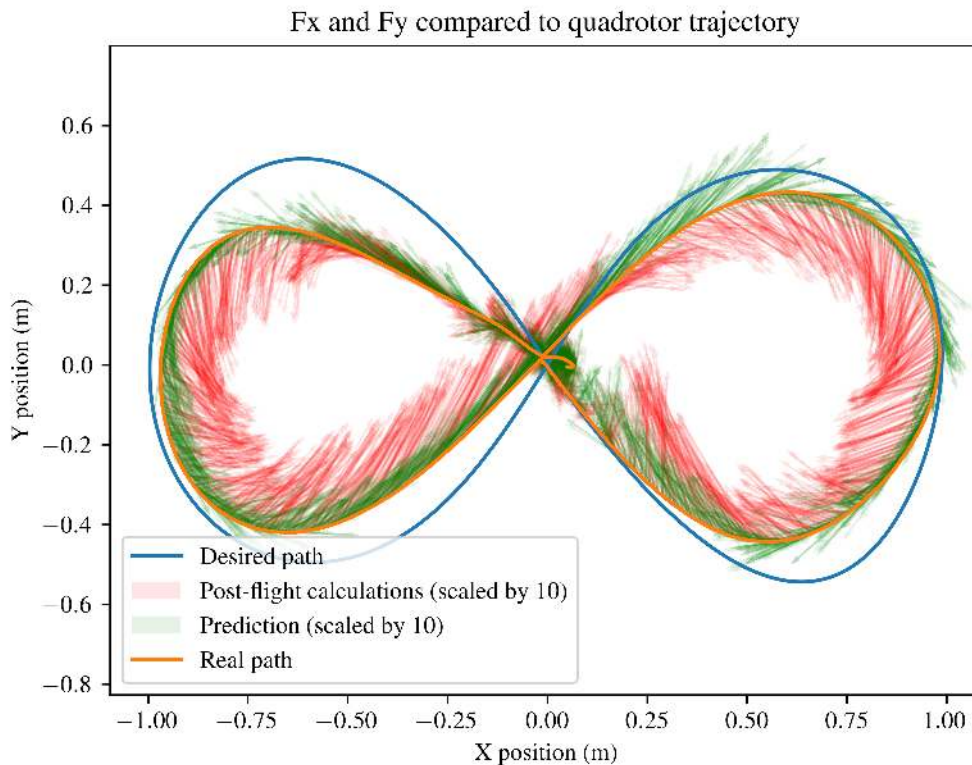


Figure 17: On-flight residual calculation of the Decision Tree Ensemble compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

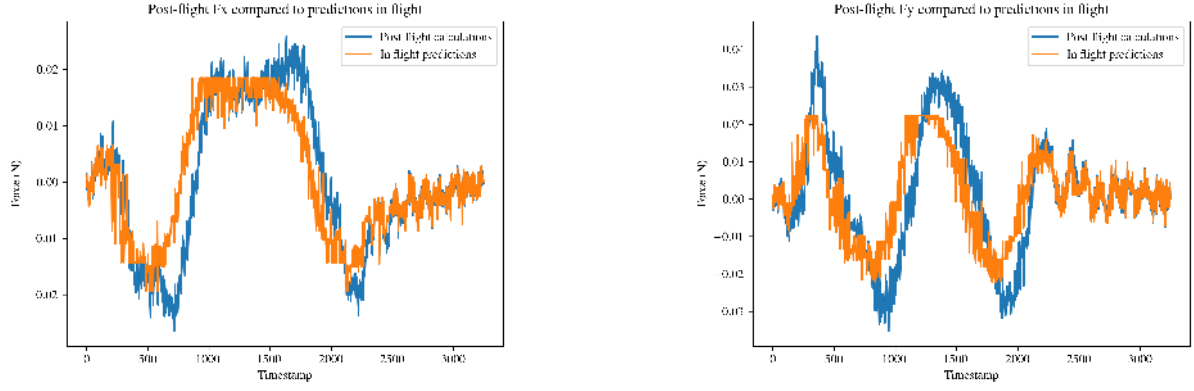


Figure 18: On-flight residual calculation of the Decision Tree Ensemble compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

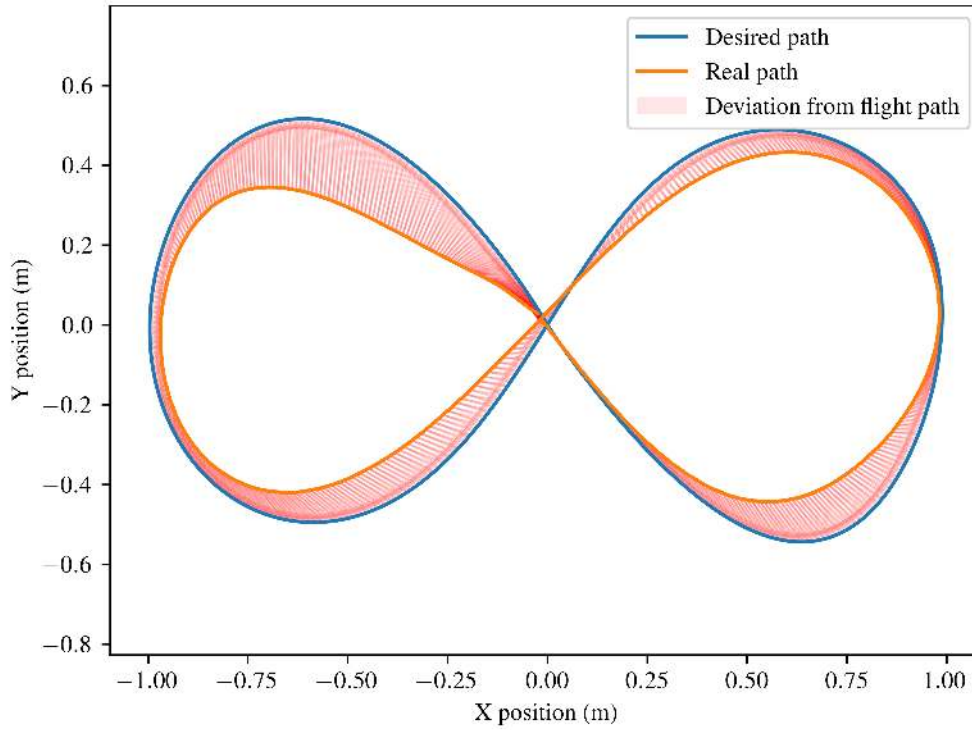


Figure 19: Deviation from the desired path using the Lee controller with the decision tree ensemble for residual force estimation on the standard Crazyflie on a 0.7 timescale flight.

5.2.4 Standard Residual Calculation Without Learning Algorithms

As can be seen in Figures 20 and 21, the predictions made post flight and during flight are perfectly aligned, this is because the standard function used is the same in both cases, they both also use the PWM values for prediction, therefore there is also no visible shift like in the DTEs or NNs.

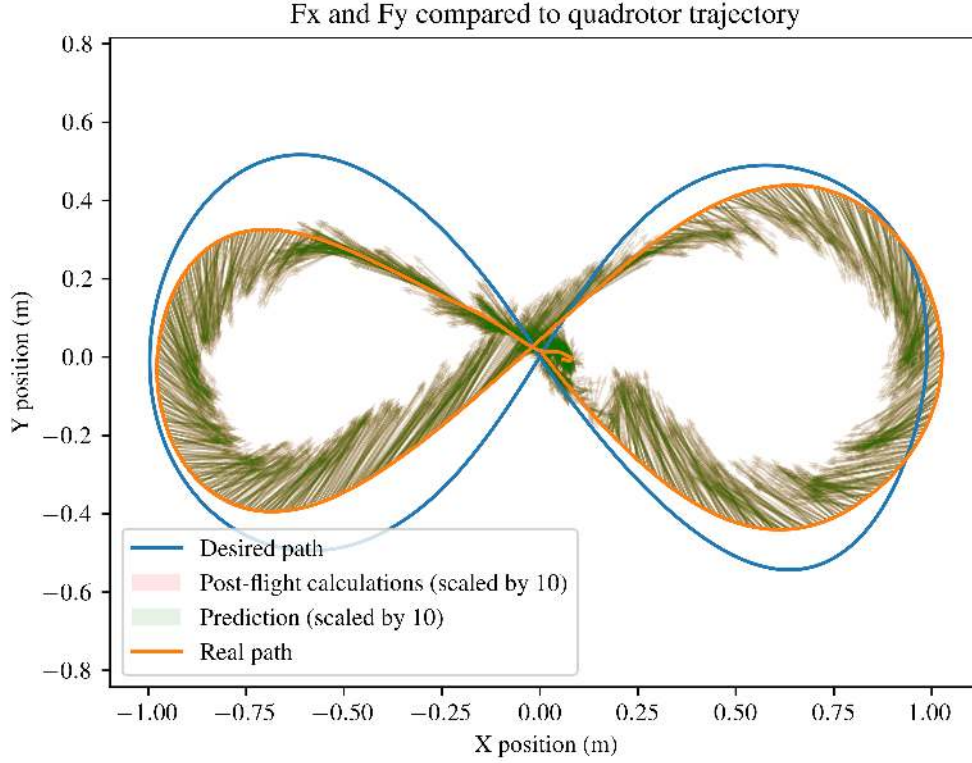


Figure 20: On-flight residual calculation of the standard method compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

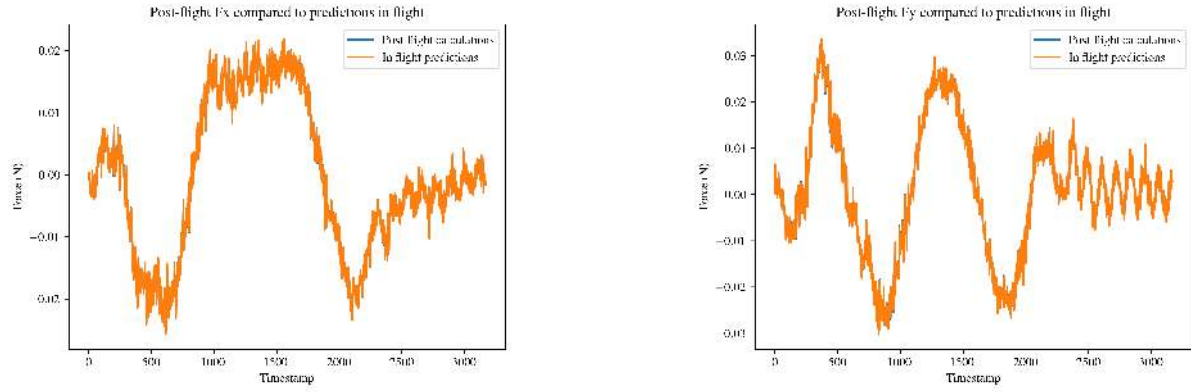


Figure 21: On-flight residual calculation of the standard method compared to the off-flight calculations on the standard Crazyflie on a 0.7 timescale flight.

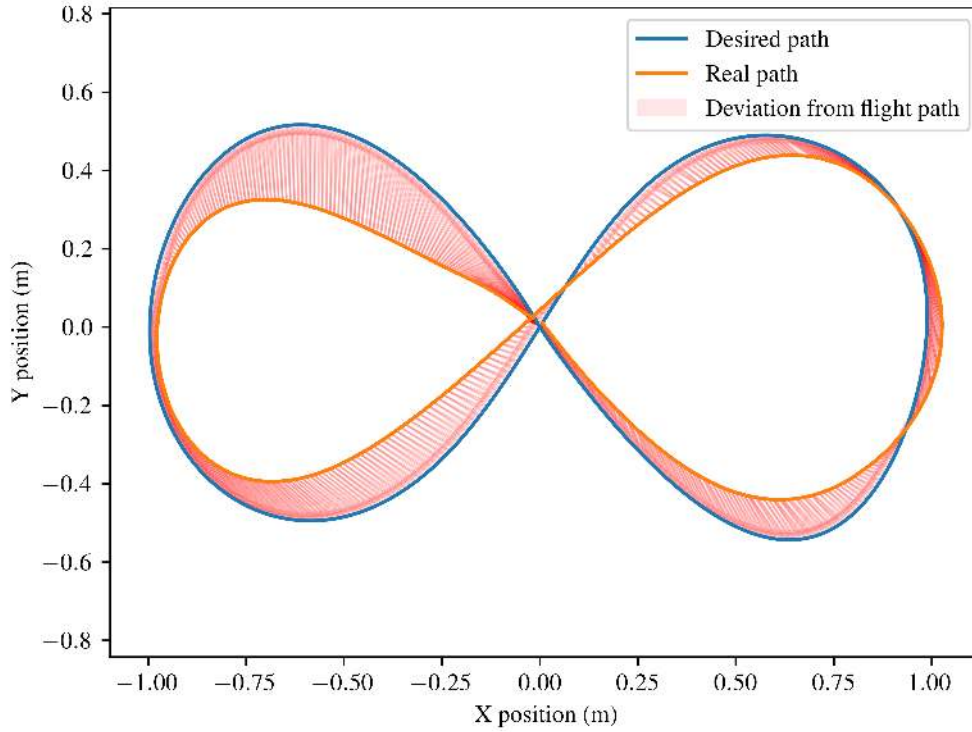


Figure 22: Deviation from the desired path using the Lee controller with the standard method for residual force estimation on the standard Crazyflie.

5.3 Results on Brushless Crazyflie

The brushless motor variant of the Crazyflie promises faster reaction times to rotor inputs and overall better flight performance this is reflected on Tables 6 and 7. The addition of the calculated residual forces on the controllers output requires very precise adjustments, so the expectation would be to see a higher percentage improvement by the residual calculations on the brushless variant.

The real world flight test where the same as on the standard Crazyflie, but with the residual predictions for the brushless Crazyflie.

Table 6: Final results on real flights with the brushless Crazyflie (each category averaged over 6 flights on figure-eight with timescale 0.7)

	avg. error (m)	extra computation time (Microseconds)
Standard Lee controller	0.08880	–
MLP	0.06599	47.4925
Decision tree	–	–
Basic calculation	0.0669	2.7962

Table 7: Final results on real flights with the brushless Crazyflie (each category averaged over 6 flights on figure-eight with timescale 0.6)

	avg. error (m)	extra computation time (Hz)
Standard Lee controller	0.1062	–
MLP	0.0876	47.4925
Decision tree	–	–
Basic calculation	0.0831	2.7962

5.3.1 Decision Tree Ensemble

In the case of the decision tree ensemble it was not possible to upload this learning model to the brushless Crazyflie. As mentioned previously the DTE has a much larger parameter count than the MLP, which makes it challenging to fit into micro-controllers with small amounts of memory. While it was possible to upload the DTE to the standard Crazyflie, it left very little space, and in the brushless version we see that this is still a problem which can make the DTE approach infeasible.

5.3.2 Neural Network

As can be seen from Figures 23 and 24, the residual predictions made by the neural network on the brushless Crazyflie seem more accurate than the ones made on the standard Crazyflie 15. This might be due to the larger amount of data that was used for training, The standard Crazyflie was trained on around 20 flights, while the brushless Crazyflie was trained on around 35 flights which were similar to the test flight but with varying timescales in the case of the brushless Crazyflie.

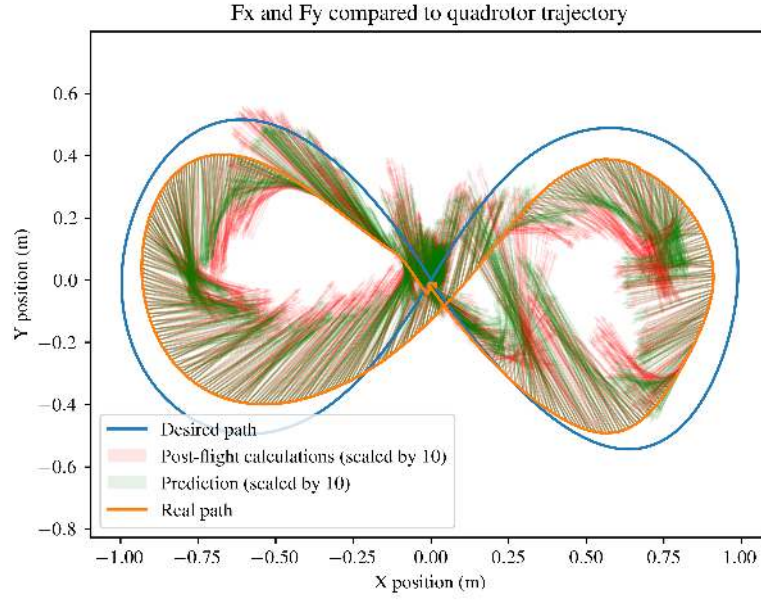


Figure 23: On-flight residual calculation of the neural network compared to the off-flight calculations on the brushless Crazyflie on a 0.6 timescale flight.

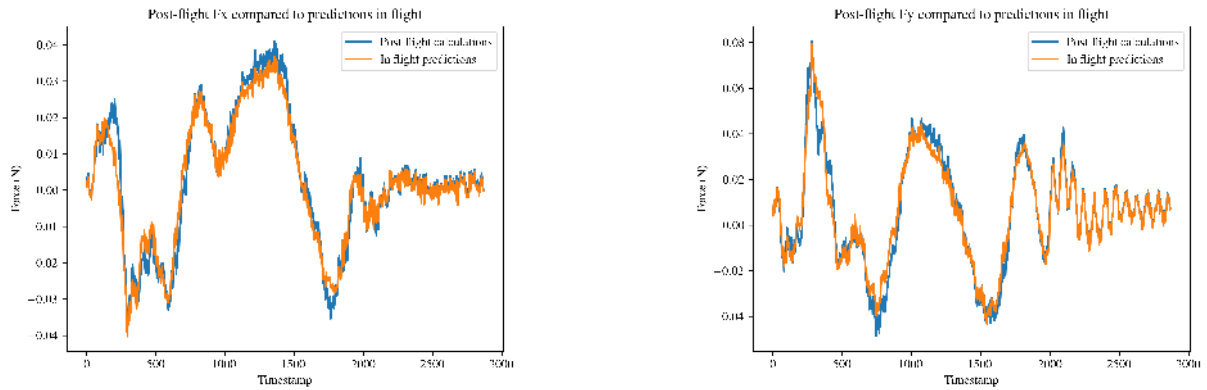


Figure 24: On-flight residual calculation of the neural network compared to the off-flight calculations on the brushless Crazyflie on a 0.6 timescale flight.

5.3.3 Standard Residual Calculation

Just like in the standard Crazyflie, we can see on Figures 25 and 26 that the standard residual calculation remains the same both on and off-board.

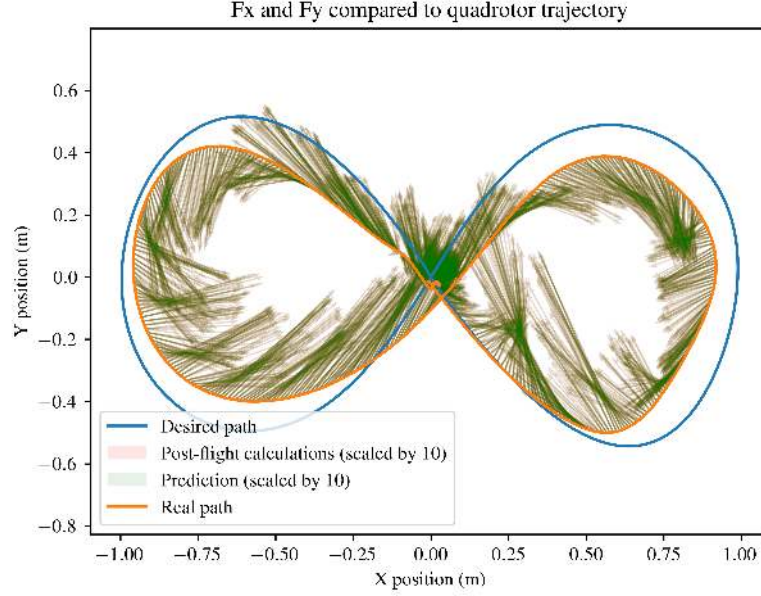


Figure 25: On-flight residual calculation of the standard method compared to the off-flight calculations on brushless Crazyflie on a 0.6 timescale flight.

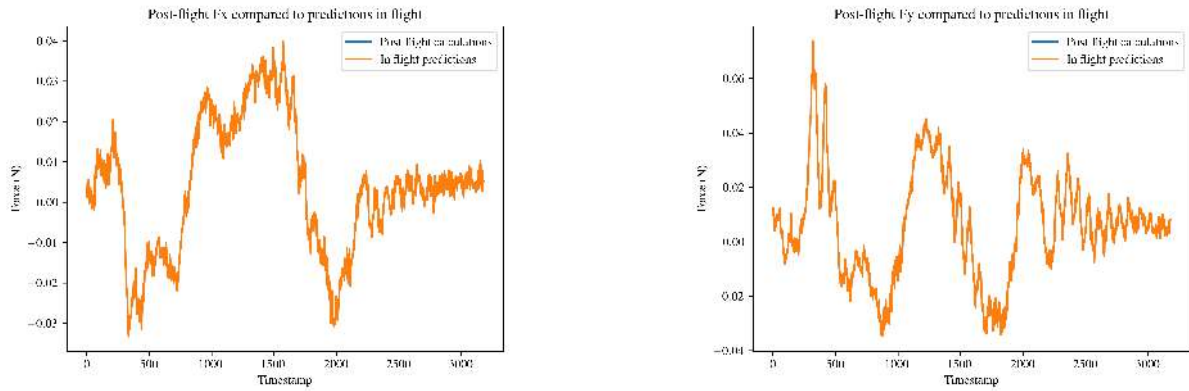


Figure 26: On-flight residual calculation of the standard method compared to the off-flight calculations on brushless Crazyflie on a 0.6 timescale flight.

5.4 Results Discussion

From the results shown in Figures 15, 18 and 24, we can see that the learning models were able to predict the residual forces acting on the quadrotors accurately. The predictions seem most accurate on the neural network trained for the brushless Crazyflie 24. This is most likely due to the fact that more training data was used for this variant of the Crazyflie, which in addition are trajectories closer to the test scene. While the MLPs seem to get better results, it's clear that the decision tree ensembles have faster inference time when compared to the neural networks.

The neural networks also seem to perform better than the standard residual calculation in most cases. This might be due to the neural networks smoothing the predictions making them overall less noisy. The decision tree ensembles, however, output what look like somewhat blocky predictions as seen in Figure 18. This might be the reason for their worse performance.

All the residual force prediction models have a better performance than using the basic Lee controller. While the performance increase is not as big as we would expect, there is still a noticeable 15% to 25% decrease in deviation from the desired flight path.

6 Future Work

In this thesis, we made a Markovian assumption about the nature of the residual forces. In the future it might be interesting to remove this assumption and calculate predictions with a sliding window approach like seen in [1].

Due to the difficulty in predicting the residual torques and the forces in the z direction, in addition to the low impact that these additional forces may have in the overall flight performance of the quadrotor, they were left out. However, future work might try to add these additional forces to our existing framework, which may lead to a more noticeable performance increase.

Another interesting avenue would be to try to predict the residual forces acting on a quadrotor carrying a payload. Little work has been done in this area which opens the door to new innovations.

The main issues encountered in this thesis had to do with the quadrotor hardware. Future work might try to use these methods on less restrictive hardware to allow for more complex flight maneuvers and bigger learning models.

7 Conclusion

It seems clear from the results seen in this thesis that there is great potential in predicting the residual forces acting on UAVs. The results show that accounting for residual forces can improve flight performance which can lead to new developments in deploying UAVs in complex tasks which require really precise flight.

This thesis also showed that neural networks can provide us with smoother residual force predictions, which can help with noisy sensor data. Another finding in this thesis is the large parameter count of decision tree ensembles. While previous work [25] was able to show better performance at predicting residual forces using this architecture, it did not look into loading these models into restrictive hardware. It's because of this that only smaller versions of DTEs were uploaded to the Crazyflie, which, while performing better inference results than its MLP counterpart on the off-line test set, outputted blocky function approximations in real world flights. In addition, a new pipeline for creating and deploying in addition to testing the models has been created. In the future it will therefore be easier to create more and better learning models to be deployed on the Crazyflie hardware.

In conclusion, deploying learning models on Crazyflie quadrotors has led to a 15% to 25% increase in performance when looking at the deviation from the desired flight path. This shows both that there are benefits to predicting the residual forces acting on UAVs as well as the viability of using certain learning models inside of the Crazyflie line of quadrotors.

References

- [1] L. Bauersfeld et al. “NeuroBEM: Hybrid Aerodynamic Quadrotor Model”. In: *Robotics: Science and Systems XVII*. Robotics: Science and Systems Foundation, 2021.
- [2] Leo Breiman. “Random Forests”. In: *Machine Learning* (2001).
- [3] Dario Brescianini, Markus Hehn, and Raffaello D’Andrea. *Nonlinear Quadcopter Attitude Control*. Tech. rep. ETH Zurich, 2013.
- [4] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016.
- [5] George Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals, and Systems* (1989).
- [6] Marc Peter Deisenroth, Dieter Fox, and Carl Edward Rasmussen. “Gaussian Processes for Data-Efficient Learning in Robotics and Control”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37.2 (2015), pp. 408–423.
- [7] Julian Förster. “System Identification of the Crazyflie 2.0 Nano Quadcopter”. Bachelor thesis. Zurich: ETH Zurich, Aug. 2015.
- [8] C. Gaz et al. “Dynamic Identification of the Franka Emika Panda Robot with Retrieval of Feasible Parameters Using Penalty-Based Optimization”. In: *IEEE Robotics and Automation Letters* (2019).
- [9] Danijar Hafner et al. “Learning Deep Dynamics Models for Planning from Pixels”. In: *arXiv preprint arXiv:1811.04551* (2019).
- [10] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural Computation* (1997).
- [11] D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. 2015.
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. 2012.
- [13] Yann LeCun et al. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* (1998).
- [14] T. Lee, M. Leok, and N. H. McClamroch. “Geometric tracking control of a quadrotor UAV on SE(3)”. In: *49th IEEE Conference on Decision and Control (CDC)*. IEEE, 2010.
- [15] Bo LI et al. “Finite-time disturbance observer-based trajectory tracking control for quadrotor unmanned aerial vehicle with obstacle avoidance”. In: *Mathematical Methods in the Applied Sciences* 46 (2022).
- [16] Wei-Yin Loh. “Classification and Regression Trees”. In: *WIREs Data Mining and Knowledge Discovery* (2011).
- [17] Robert Mahony, Vijay Kumar, and Peter Corke. “Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor”. In: *IEEE Robotics & Automation Magazine* (2012).
- [18] D. Mellinger and V. Kumar. “Minimum Snap Trajectory Generation and Control for Quadrotors”. In: *IEEE International Conference on Robotics and Automation*. 2011.

- [19] Vinod Nair and Geoffrey E. Hinton. “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)* (2010).
- [20] Open Source Robotics Foundation. *Robot Operating System (ROS)*. <https://www.ros.org/>. 2007.
- [21] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. URL: <https://arxiv.org/abs/1912.01703>.
- [22] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* (2011). URL: <http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>.
- [23] J. Ross Quinlan. “Induction of Decision Trees”. In: *Machine Learning* (1986).
- [24] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* (1986).
- [25] J. Schicke. “Learning-based Multirotor System Model Enhancements”. Examiners: Wolfgang Hönig, Marc Toussaint; Mentors: Akmaral Moldagalieva, Khaled Wahba. Bachelor thesis. 2023.
- [26] I. Schubert et al. “A Generalist Dynamics Model for Control”. In: *arXiv:2305.10912* (2023).
- [27] G. Shi et al. “Neural-Swarm2: Planning and Control of Heterogeneous Multirotor Swarms Using Learned Interactions”. In: *IEEE Transactions on Robotics* (2021).
- [28] Guanya Shi et al. “Neural-Swarm: Decentralized Close-Proximity Multirotor Control Using Learned Interactions”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2020.
- [29] H. T. Siegelmann, B. G. Horne, and C. L. Giles. “Computational capabilities of recurrent NARX neural networks”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* (1997).
- [30] Ewoud J.J. Smeur, Qiping Chu, and Guido C.H.E. de Croon. “Adaptive Incremental Nonlinear Dynamic Inversion for Attitude Control of Micro Aerial Vehicles”. In: *IEEE Conference on Control Applications (CCA)*. 2016.
- [31] Ashish Vaswani et al. “Attention is all you need”. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*. 2017.
- [32] Jerry Ye et al. “Stochastic Gradient Boosted Distributed Decision Trees”. In: *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM '09)*. Association for Computing Machinery, 2009.